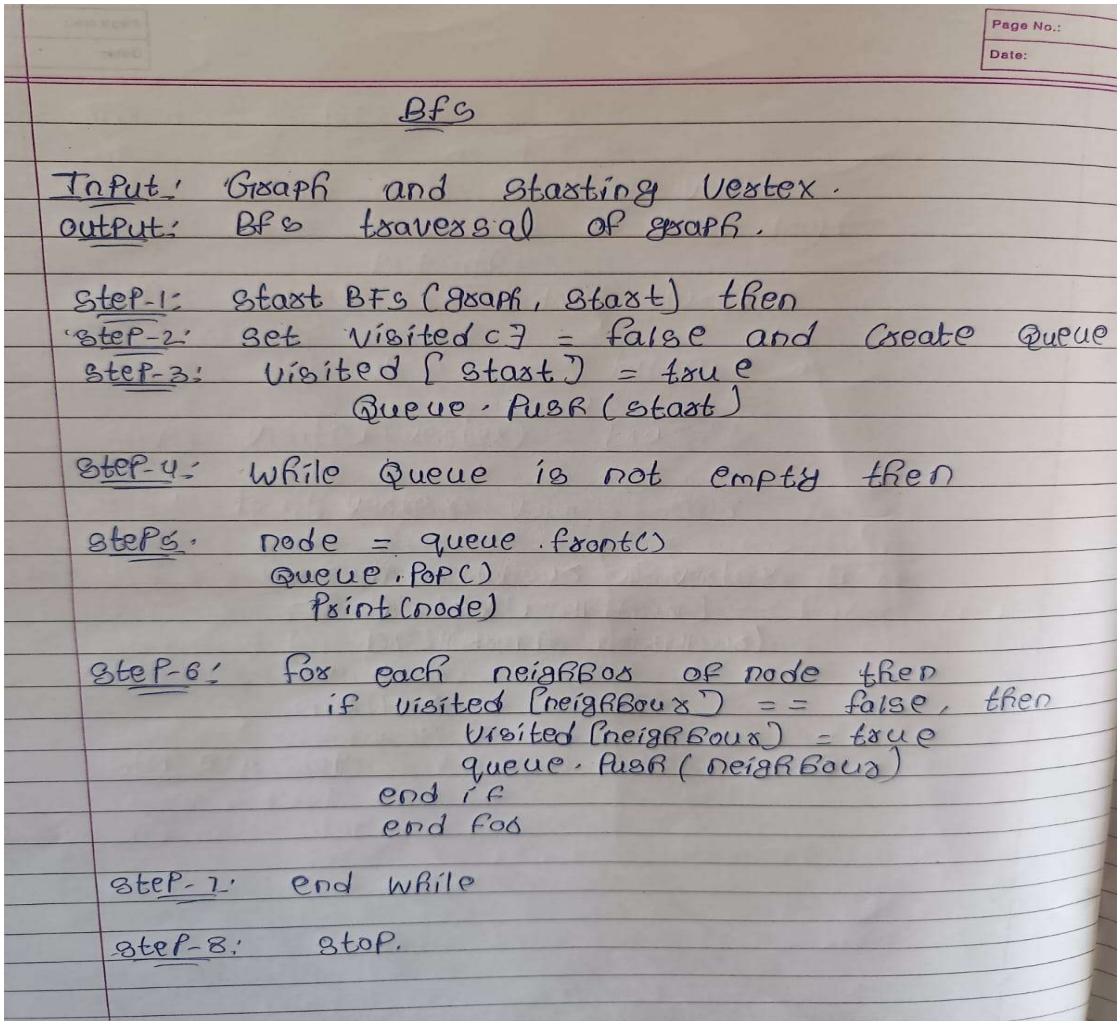
PRACTICAL-08

AIM: Implementation of Graph and Searching (DFS and BFS).

1.) BFS:

ALGORITHM:



BFS

Input: Graph and starting vertex.
output: BFS traversal of graph.

step-1: start BFS (graph, start) then
step-2: set visited [] = false and create Queue
step-3: visited [start] = true
Queue.push(start)

step-4: while Queue is not empty then

step-5: node = queue.front()
Queue.pop()
Print(node)

step-6: for each neighbors of node then
if visited [neighbour] == false, then
visited [neighbour] = true
queue.push(neighbour)
end if
end for

step-7: end while

step-8: stop.

PROGRAM:

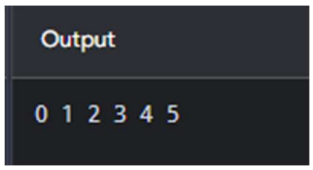
```
#include <iostream>

#include <vector>
```

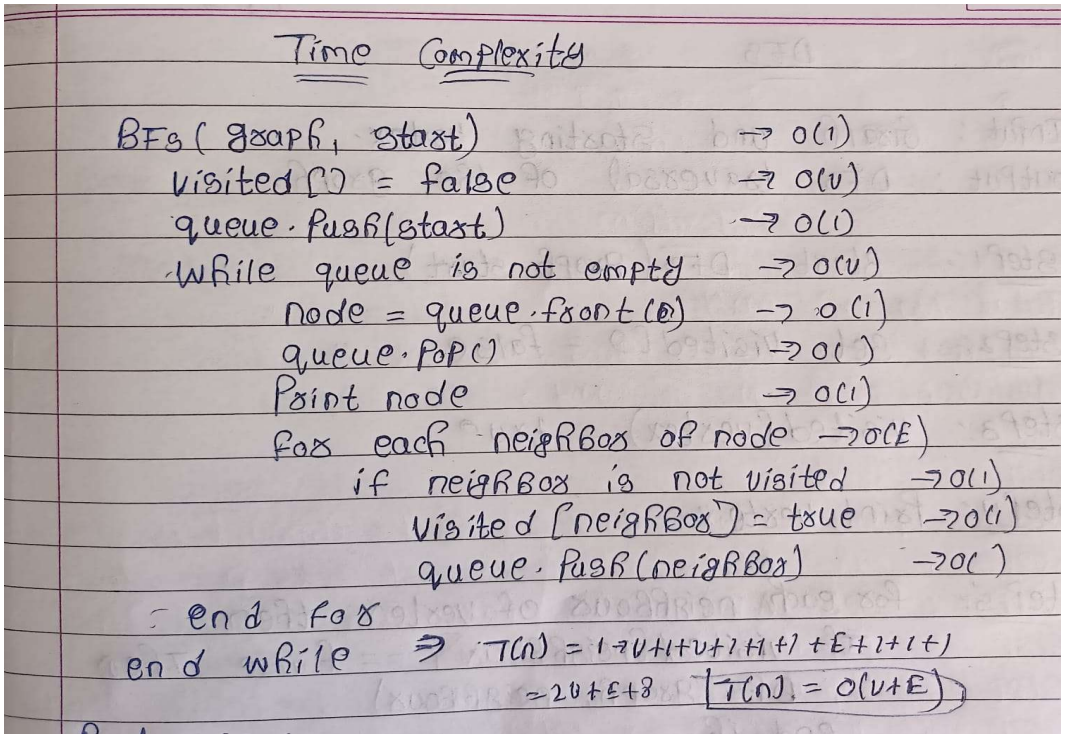
```
#include <queue>
using namespace std;
void BFS(vector<vector<int>> graph, int start)
{
    bool visited[6] = {false};
    queue<int> q;
    visited[start] = true;
    q.push(start);
    while (!q.empty())
    {
        int node = q.front();
        q.pop();
        cout << node << " ";
        for (int x : graph[node])
        {
            if (!visited[x])
            {
                visited[x] = true;
                q.push(x);
            }
        }
    }
}
int main()
{
    vector<vector<int>> graph = {
        {1, 2},
        {0, 3, 4},
        {0, 5},
```

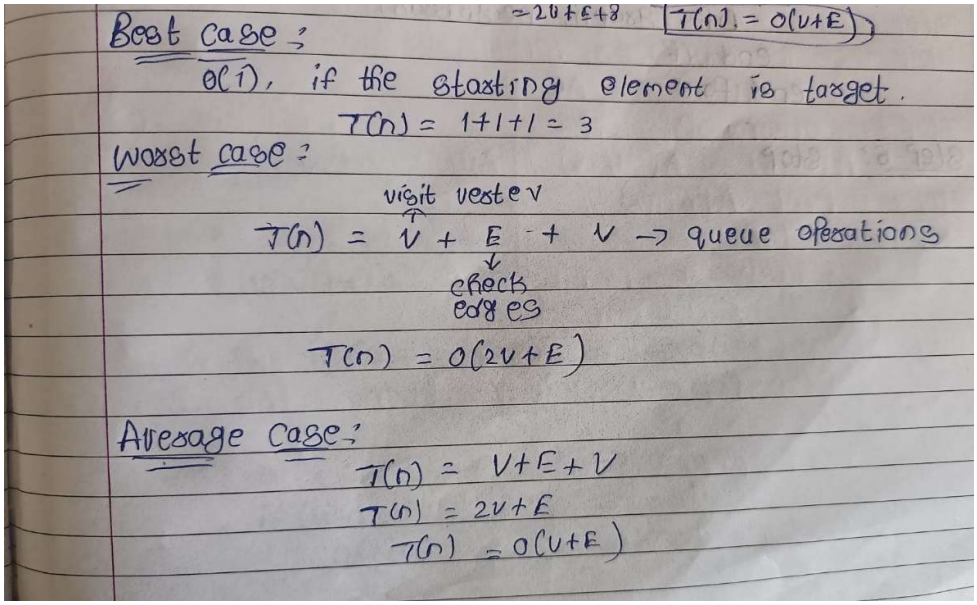
```
    {1},
    {1},
    {2}
  };
  BFS(graph, 0);
  return 0;
}
```

OUTPUT:



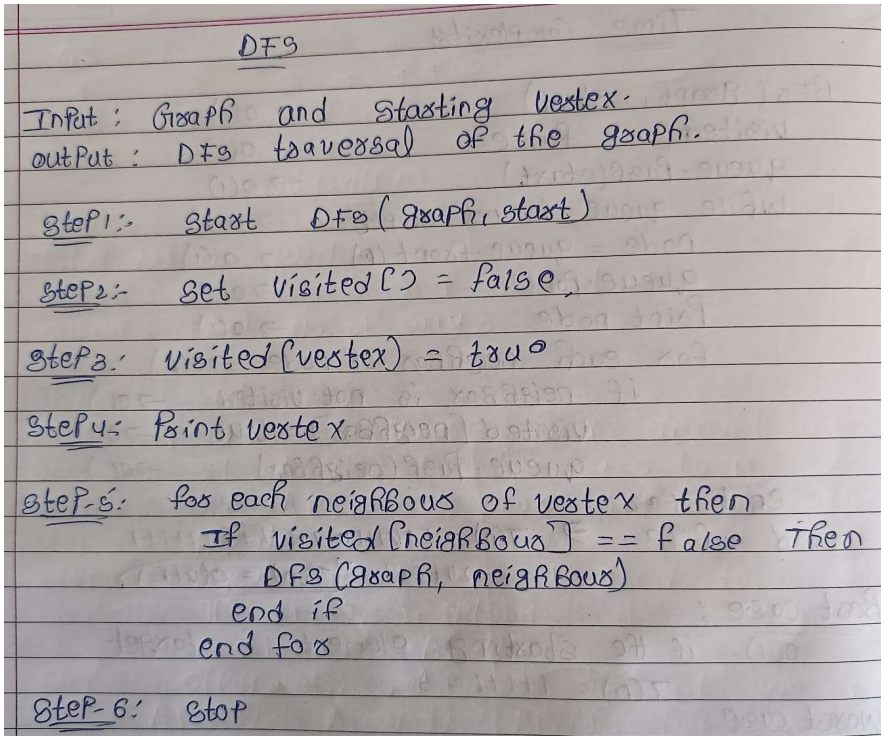
TIME COMPLEXITY:





2.) DFS

ALGORITHM:



PROGRAM:

```
#include <iostream>
#include <vector>
using namespace std;
void DFS(vector<vector<int>> graph, int vertex, bool visited[])
{
   visited[vertex] = true;
   cout << vertex << " ";
   for (int neighbor : graph[vertex])
   {
      if (!visited[neighbor])
      {
         DFS(graph, neighbor, visited);
      }
   }
}

int main()
{
   vector<vector<int>> graph = {

      {1, 2},
      {0, 3, 4},
      {0, 5},
      {1},
      {1},
```

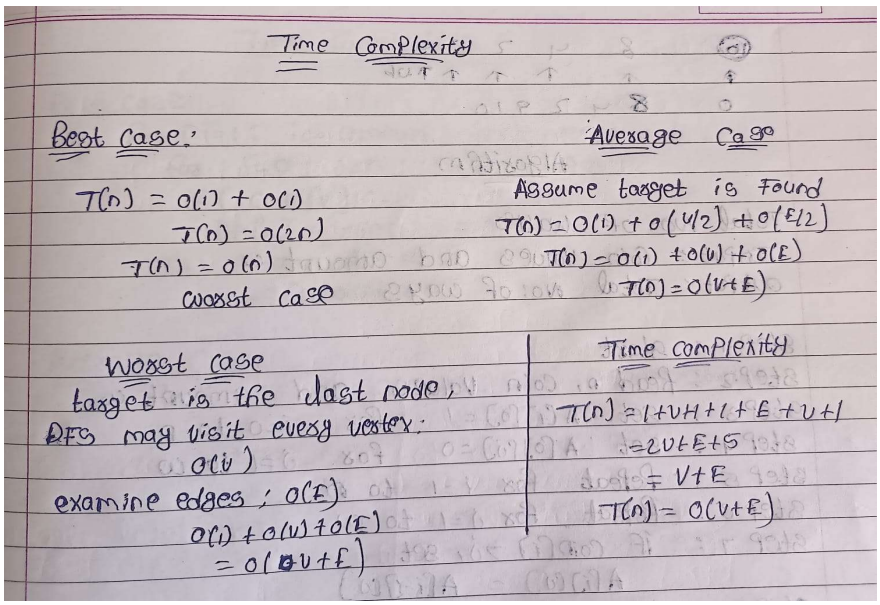
```
    {2}
  };
  bool visited[6] = {false};
  DFS(graph, 0, visited);
  return 0;
}
```

OUTPUT:

```
Output

0 1 3 4 2 5
```

TIME COMPLEXITY:



CONCLUSION:

BFS and DFS are graph traversal techniques used to visit all the vertices of a graph. BFS uses a queue to visit vertices level by level, while DFS uses recursion/stack to explore deeply. Both are useful for solving different graph problems and have O(V + E) time complexity using an adjacency list.